Case Study: Re-architecting a Technician Resource Portal

Stack: Go, AWS Lambda, AWS EventBridge, AWS S3, GitHub Actions, Google Drive API, Terraform, Prometheus, Grafana

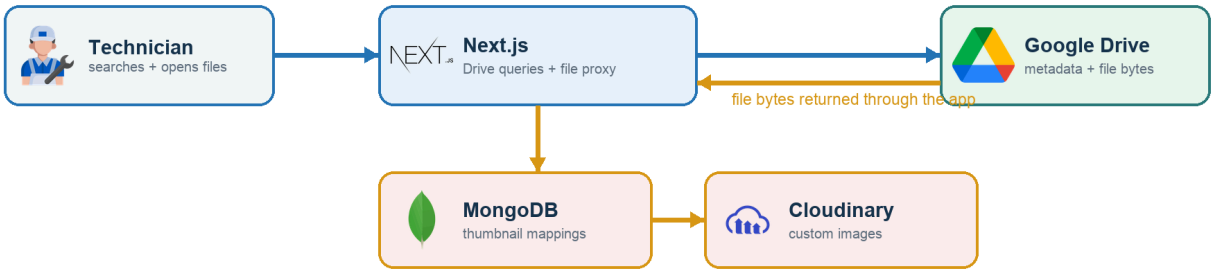
The Challenge: Scaling Document Access

otasupport.com publishes thousands of Google Drive files for technicians. Originally, the shared Drive experience was difficult to search. Relying on manual permissions was an operational bottleneck that scaled poorly as the team and library grew.

To solve this, the first custom website was built. While it successfully provided a dedicated portal for technicians, the underlying architecture struggled to scale efficiently with user demand and eventually grew beyond the initial problem.

Legacy Architecture: Client Fetches Catalog

https://v1.otasupport.com



The initial application fetched data from Google Drive dynamically on every refresh. It proxied file downloads through Vercel and maintained thumbnail mappings using MongoDB and Cloudinary.

As traffic grew to 4,573 visitors and 13,200 page views over a 12-month period, this architecture revealed several critical flaws:

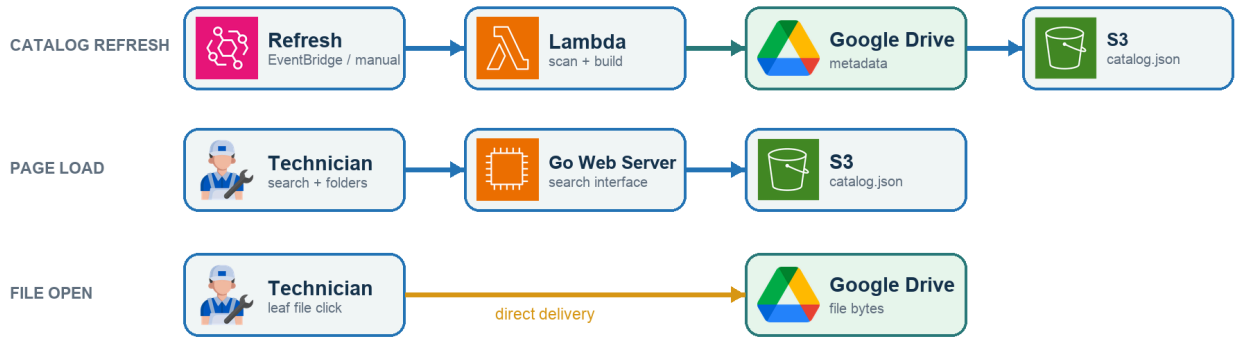
- **Unsustainable Bandwidth:** Proxying files added significant application bandwidth and runtime pressure. Over 12 months, this generated 639 GB[1] of outbound transfer and over 1 million edge requests.

[1] https://drive.google.com/file/d/1XMOk7Zc_9tosxqHrZ3B639i1dLHm8GhS/view?usp=drive_link

- **Delivery Failures:** Because the application sat directly in the delivery path, large PDF files would frequently fail to load.
- **Performance Bottlenecks:** Because the application sat as a live middleware, it queried Google Drive's API on the fly for every user request. This was inherently slow and resource-heavy, causing significant lag before technicians could even begin searching for files.
- **Complex Content Management:** Managers had to follow complicated content rules[2]. For example, they had to wrap text in parentheses to prevent it from displaying on the UI, and keep unpublished files strictly in the Root Folder.

Newer Architecture: Server Precomputes Catalog

https://otasupport.com



Recognizing that the core requirement was simply to index the library rather than relay every file, the system was re-architected using Go and AWS. The guiding principle for V2 was to offload all catalog generation to a background service, eliminating the need to query Google Drive on the fly

- **Decoupled Catalog Generation:** Catalog generation was separated from customer traffic. An AWS Lambda function, triggered by EventBridge once a day (or via a manual refresh), scans Drive and updates the metadata catalog in private S3.
- **Direct Delivery:** A Go application running on EC2 serves a simple web interface. Search runs directly in the browser, and leaf files open directly from Google Drive. This completely bypasses the application's bandwidth.

[2] https://drive.google.com/file/d/1NRU3g72RnCBlnRvOTcQ3ZR2cRFcW-fek/view?usp=drive_link

- **Simplified Management:** Google Drive is kept as the source of truth, allowing managers to continue using familiar folders without copying any customer files to AWS.

Impact & System Comparison

By redesigning the application around a precomputed catalog rather than live proxying, the system achieved drastic improvements in speed, bandwidth efficiency, and architectural simplicity.

Area	V1 (Next.js Middleware)	V2 (Go Search-First Catalog)
Customer Request	Live Drive work	Prebuilt S3 catalog
File Delivery	Proxied by application	Direct from Google Drive
Supporting Systems	Next.js + MongoDB + Cloudinary	Go service + one JSON snapshot
Catalog Response	About 3 seconds	About 0.32 seconds
Outbound Transfer	639.91 GB / 12 months	About 1.65GB[3]

V1 treated the application server as a pass-through proxy for Google Drive binary data. V2 re-framed the server as an indexer—storing a tiny, static JSON snapshot in S3—and delegated file delivery directly to Google Drive. This eliminated massive application-level bandwidth costs, completely resolved large PDF download timeouts, and reduced catalog response times by nearly 10x.

Observability & Monitoring[4]

To gain insight into system health and technician usage, observability was added using Grafana, Prometheus, and CloudWatch. Rather than introducing the complexity of Kubernetes, the monitoring tools run via Docker Compose alongside the Go web server on the existing EC2 host. The system tracks core engagement events—such as page views, folder navigation, and file opens—while strictly excluding sensitive data like search queries or authorization headers.

---

[3] 13,200 (annual page views) * 121KB (catalog file size) ≈ 1.6GB
[4] http://otasupport.com/admin/monitoring